

Algorithm Design

Learning Objectives

- Break down complex problems into smaller, manageable steps
- Use pseudocode and flowcharts to design algorithms before coding
- Evaluate algorithm efficiency using time and space complexity concepts

Engage (5-7 minutes)

Before constructing a building, an architect creates a detailed blueprint. Similarly, before writing code, a programmer designs an algorithm—a step-by-step plan to solve the problem. Good algorithms save time, reduce errors, and make code easier to understand. Today we learn to think before we code and design solutions systematically.

Explore (10-12 minutes)

Given the problem “find the second largest number in an unsorted array,” brainstorm at least two different approaches with a partner. Write each approach as a numbered list of steps. Which approach uses fewer steps? Which is easier to implement? Now try to express one approach as a flowchart using boxes and arrows.

Explain (10-12 minutes)

Algorithm design is the process of defining a clear, step-by-step procedure to solve a problem. Techniques include decomposition (breaking problems into subproblems), pattern recognition, abstraction (focusing on essential details), and algorithmic thinking. Pseudocode uses plain English mixed with programming constructs to outline logic. Flowcharts visualize the flow with standard symbols: ovals for start/end, rectangles for processes, diamonds for decisions, and arrows for flow direction.

Elaborate (8-10 minutes)

Design algorithms for: finding the GCD of two numbers using Euclid’s algorithm, checking if a number is prime, and sorting an array using insertion sort. Write each as pseudocode first, then draw a flowchart for the prime-checking algorithm. Analyze the time complexity of each algorithm using Big-O notation. Discuss with a class: what makes an algorithm “good”—is it speed, simplicity, or memory usage?

Evaluate (5-8 minutes)

Write pseudocode for an algorithm that takes a list of exam scores and determines how many students scored above, below, or at the class average. Create a flowchart for the same algorithm. Explain why designing the algorithm before coding prevents bugs and saves development time.